

DỰ ÁN HỌC MÁY TỪ ĐẦU ĐẾN CUỐI

Chương 2 - End-to-End Machine Learning Project

TS. Trần Thành Thắng

Đại học Đông Á

August 5, 2026

Nội dung Chương trình

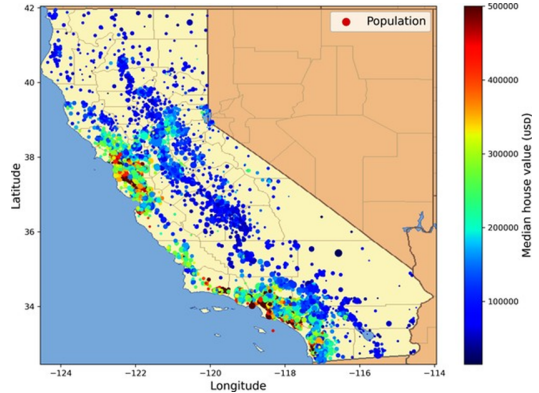
1. Giới thiệu Dự án & Khởi tạo Dữ liệu
2. Khám phá & Chuẩn bị Dữ liệu
3. Chọn, Huấn luyện, Tinh chỉnh & Triển khai

- Hiểu rõ toàn bộ quy trình phát triển một dự án học máy thực tế từ khi bắt đầu đến khi triển khai.
- Thực hành các kỹ thuật chuẩn bị, làm sạch, và biến đổi dữ liệu.
- Trực quan hóa dữ liệu để tìm ra các mẫu (patterns) và tương quan.
- Xây dựng pipeline dữ liệu chuẩn.
- Đánh giá, tinh chỉnh và lựa chọn mô hình học máy tốt nhất.

1. Giới thiệu Dự án & Khởi tạo Dữ liệu

Làm việc với dữ liệu thực

- Học máy tốt nhất là thực hành trên dữ liệu thực.
- Trong chương này, chúng ta sử dụng tập dữ liệu **Nhà ở California (California Housing Prices)**.
- Mục tiêu: Dự đoán giá nhà trung bình tại một khu vực bất kỳ ở California dựa trên các đặc trưng như dân số, thu nhập trung bình, v.v.

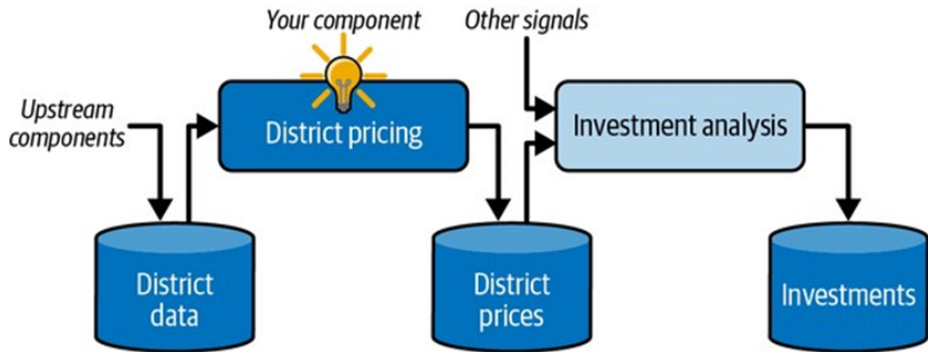


Hình 2-1. Giá nhà California

Nhìn vào bức tranh lớn: Xác định vấn đề

- Trước khi phân tích, cần hiểu mục đích kinh doanh là gì? Mô hình này sẽ được sử dụng như thế nào?
- Việc hiểu mục đích sẽ giúp quyết định: Thuật toán nào? Thước đo hiệu suất nào? Mức độ đầu tư công sức?
- **Bài toán của chúng ta:** Hệ thống nhận dữ liệu khu vực và dự đoán giá nhà. Đầu ra này sẽ được đưa vào một hệ thống máy học khác để quyết định có nên đầu tư vào khu vực đó hay không.

Pipeline hệ thống học máy



Hình 2-2. Một pipeline máy học cho đầu tư bất động sản

Chọn một Thước đo Hiệu suất

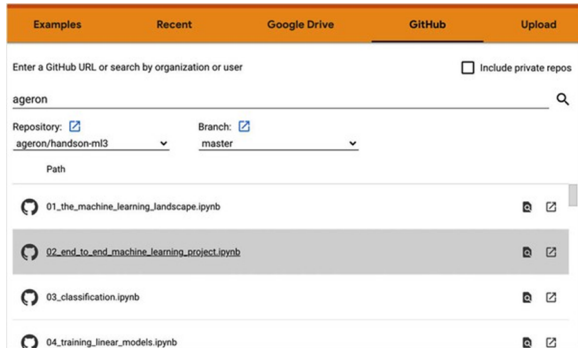
- Vì đây là bài toán hồi quy (Regression), chúng ta sử dụng **Căn bậc hai Sai số Toàn phương Trung bình (RMSE - Root Mean Square Error)**.
- RMSE cho biết sai số dự đoán điển hình. Càng nhỏ càng tốt.
- Nếu dữ liệu có nhiều ngoại lai (outliers), ta có thể sử dụng **Sai số Tuyệt đối Trung bình (MAE - Mean Absolute Error)**.

- Luôn liệt kê và xác minh các giả định đã đặt ra.
- Ví dụ: Hệ thống hạ nguồn có thực sự cần dự đoán giá trị cụ thể, hay chỉ cần các hạng mục (rẻ, trung bình, đắt)?
- Nếu họ chỉ cần hạng mục, thì chúng ta nên cấu trúc nó thành bài toán Phân loại (Classification) thay vì Hồi quy.

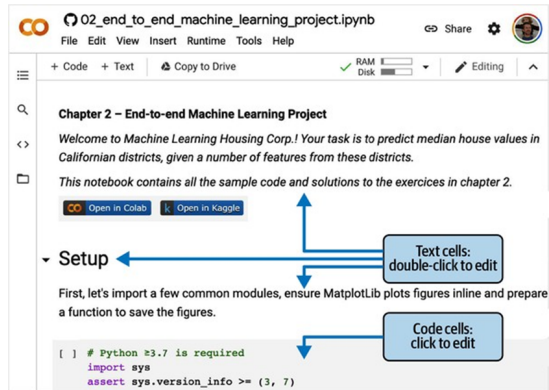
Lấy Dữ liệu & Môi trường làm việc

- Chúng ta sẽ sử dụng **Google Colab** - một môi trường Jupyter Notebook miễn phí trên đám mây.
- Không cần cài đặt phần mềm phức tạp trên máy tính cá nhân.
- Cung cấp GPU/TPU miễn phí để huấn luyện mô hình.

Minh họa: Môi trường Google Colab



Hình 2-3. Giao diện Google Colab



Hình 2-4. Chạy code trên Colab

```
housing.head()
```

	longitude	latitude	housing_median_age	median_income	ocean_proximity	median_house_value
0	-122.23	37.88	41.0	8.3252	NEAR BAY	452600.0
1	-122.22	37.86	21.0	8.3014	NEAR BAY	358500.0
2	-122.24	37.85	52.0	7.2574	NEAR BAY	352100.0
3	-122.25	37.85	52.0	5.6431	NEAR BAY	341300.0
4	-122.25	37.85	52.0	3.8462	NEAR BAY	342200.0

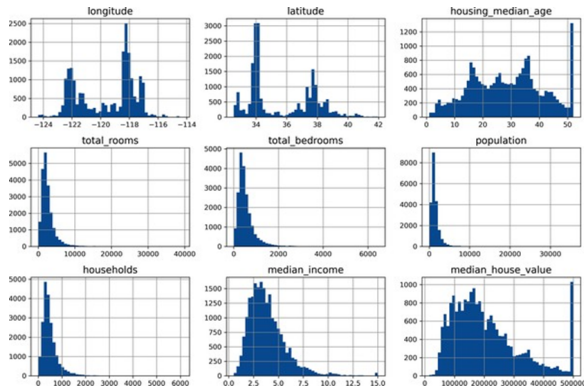
Hình 2-6. Năm hàng đầu tiên trong tập dữ liệu

Xem tóm tắt dữ liệu

```
housing.describe()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	median_house_value
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	206855.816909
std	2.003532	2.135952	12.585558	2181.615252	421.385070	115395.615874
min	-124.350000	32.540000	1.000000	2.000000	1.000000	14999.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	119600.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	179700.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	264725.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	500001.000000

Hình 2-7. Tóm tắt của từng thuộc tính số



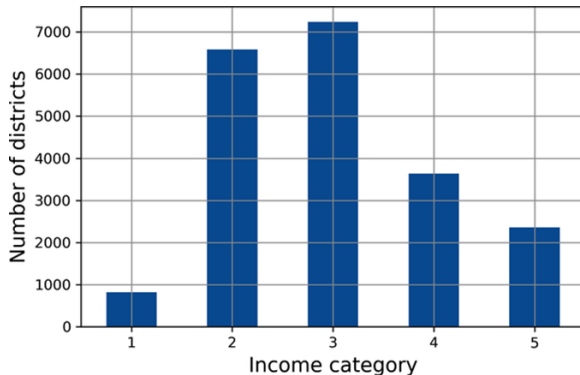
Hình 2-8. Biểu đồ tần suất (Histogram)

Tạo tập kiểm thử (Test set): Tại sao cần thiết?

- Chúng ta phải tách một phần dữ liệu (khoảng 20%) ngay lập tức và **không bao giờ nhìn vào nó**.
- Lý do: Tránh *thiên kiến nhìn mò dữ liệu (Data Snooping Bias)*.
- Nếu chọn thuật toán dựa trên toàn bộ dữ liệu, mô hình có thể Overfit trên tập đó nhưng dự đoán kém trong thực tế.

Phân phối Thu nhập

- Thu nhập trung bình là một thuộc tính rất quan trọng để dự đoán giá nhà.
- Nếu lấy mẫu ngẫu nhiên thuần túy, ta có thể vô tình chọn phải một tập kiểm thử thiếu tính đại diện.



Hình 2-9. Các loại thu nhập

Lấy mẫu phân tầng (Stratified Sampling) so với Ngẫu nhiên

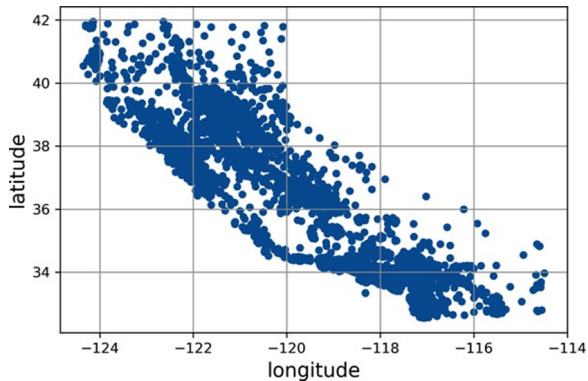
	Overall %	Stratified %	Random %	Strat. Error %	Rand. Error %
Income Category					
1	3.98	4.00	4.24	0.36	6.45
2	31.88	31.88	30.74	-0.02	-3.59
3	35.06	35.05	34.52	-0.01	-1.53
4	17.63	17.64	18.41	0.03	4.42
5	11.44	11.43	12.09	-0.08	5.63

Hình 2-10. Tỷ lệ phân loại thu nhập trong tổng thể so với mẫu ngẫu nhiên và phân tầng

2. Khám phá & Chuẩn bị Dữ liệu

Trực quan hóa dữ liệu địa lý

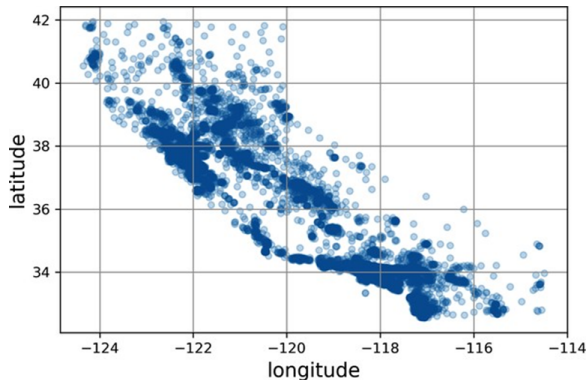
- Dữ liệu có thuộc tính kinh độ (longitude) và vĩ độ (latitude).
- Việc vẽ biểu đồ phân tán (scatter plot) sẽ giúp ta nhìn thấy mô hình phân bố của các khu dân cư.



Hình 2-11. Trực quan hóa dữ liệu địa lý

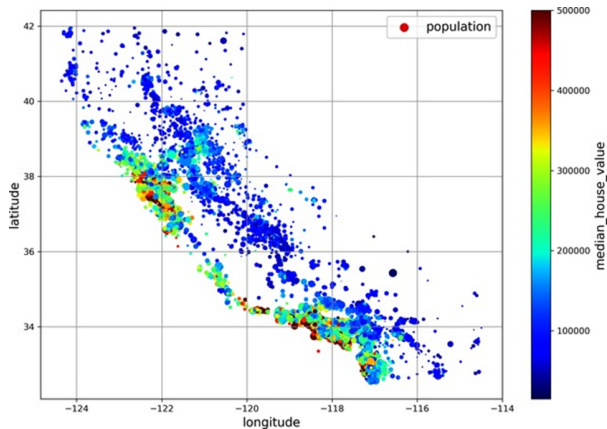
Phân tích mật độ địa lý

- Bằng cách chỉnh thông số alpha, ta có thể nhìn thấy rõ ràng các khu vực tập trung đông dân cư.
- Nổi bật lên khu vực Vùng Vịnh (Bay Area), Los Angeles và San Diego.



Hình 2-12. Trực quan hóa tốt hơn để thấy khu vực mật độ cao

Giá nhà theo dân số & Vị trí

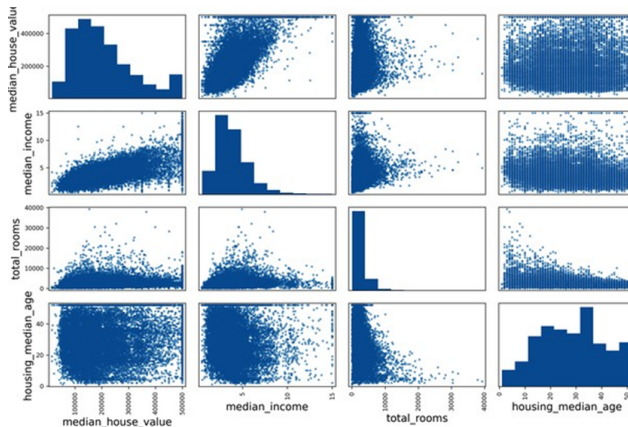


Hình 2-13. Giá nhà khu vực California (Màu: Giá nhà, Vòng tròn: Dân số)

Tìm kiếm các tương quan (Correlation)

- Hệ số tương quan Pearson (r) chạy từ -1 đến +1.
- Gần +1: Tương quan dương mạnh (ví dụ: thu nhập tăng, giá nhà tăng).
- Gần -1: Tương quan âm mạnh.
- Gần 0: Không có tương quan tuyến tính.

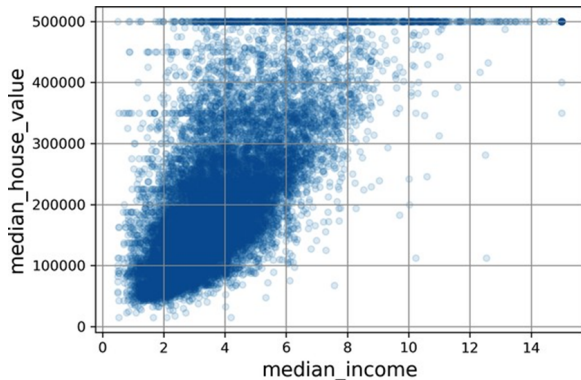
Ma trận phân tán (Scatter Matrix)



Hình 2-14. Ma trận phân tán hiển thị tương quan giữa các đặc trưng

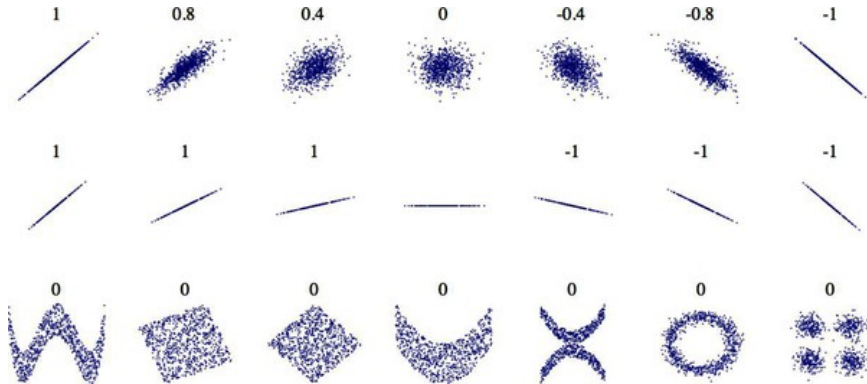
Thu nhập trung bình vs Giá nhà trung bình

- Thu nhập trung bình là thuộc tính có tương quan mạnh nhất với giá nhà.
- Trên biểu đồ xuất hiện một số "đường ngang" (trần giá) ở mức \$500k, \$450k, và \$350k.
- Có thể cần phải loại bỏ các điểm dữ liệu này để thuật toán không học những mô hình sai lệch này.



Hình 2-15. Thu nhập so với giá trị nhà

Phân phối và độ lệch chuẩn của dữ liệu



Hình 2-16. Độ lệch chuẩn của các tập dữ liệu khác nhau

Thử nghiệm với các kết hợp đặc trưng

- Đôi khi, kết hợp các đặc trưng hiện có sẽ tạo ra đặc trưng mới ý nghĩa hơn.
- **Ví dụ:**
 - Số lượng phòng trong một quận không hữu ích nếu không biết số hộ gia đình. Thay vào đó, tạo đặc trưng *Số phòng trên mỗi hộ*.
 - Số lượng phòng ngủ trên tổng số phòng.
 - Dân số trên mỗi hộ.

Chuẩn bị dữ liệu cho học máy

Làm sạch dữ liệu (Data Cleaning)

- Các thuật toán học máy không thể hoạt động với các giá trị bị thiếu (Missing Values).
- Có 3 lựa chọn để xử lý:
 - 1 Loại bỏ các mẫu tương ứng (xóa hàng).
 - 2 Loại bỏ toàn bộ thuộc tính đó (xóa cột).
 - 3 Điền vào các giá trị bị thiếu bằng một giá trị nào đó (0, trung bình, trung vị...).
- Sử dụng SimpleImputer của Scikit-Learn.

Xử lý các thuộc tính văn bản và phân loại

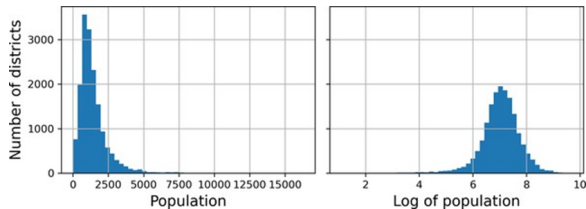
- Hầu hết các thuật toán ML thích làm việc với con số. Cần chuyển đổi văn bản sang số.
- Đặc trưng "ocean_proximity" là văn bản.
- Phương pháp **One-Hot Encoding**:
 - Tạo một cột nhị phân cho mỗi danh mục.
 - Tránh việc thuật toán hiểu sai rằng danh mục "1" thì gần danh mục "2" hơn danh mục "4".

Co giãn Đặc trưng (Feature Scaling)

- Các thuật toán hoạt động không tốt khi các đặc trưng có thang đo rất khác nhau (ví dụ: dân số từ vài nghìn, nhưng thu nhập chỉ từ 0 - 15).
- **Min-Max Scaling (Chuẩn hóa Normalization):** Giá trị được chuyển về khoảng từ 0 đến 1.
- **Standardization (Chuẩn hóa Standardization):** Trừ đi giá trị trung bình và chia cho độ lệch chuẩn. Không bị ảnh hưởng nhiều bởi ngoại lai.

Biến đổi Đặc trưng để gần với phân phối Gaussian

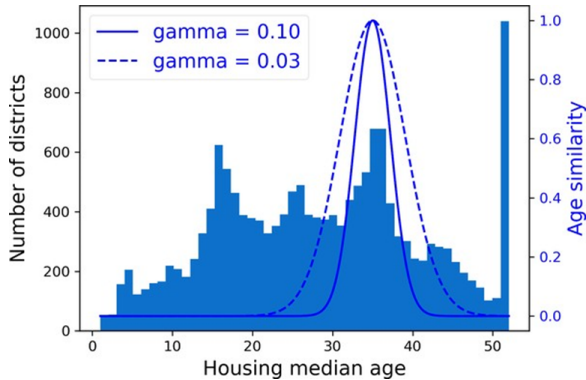
- Nhiều biểu đồ tần suất bị "nặng đuôi" (tail-heavy).
- Sử dụng các phép tính logarit (Log) hoặc logarit căn bản để biến đổi dữ liệu về dạng giống cái chuông (bell-shaped Gaussian).



Hình 2-17. Biến đổi đặc trưng nặng đuôi

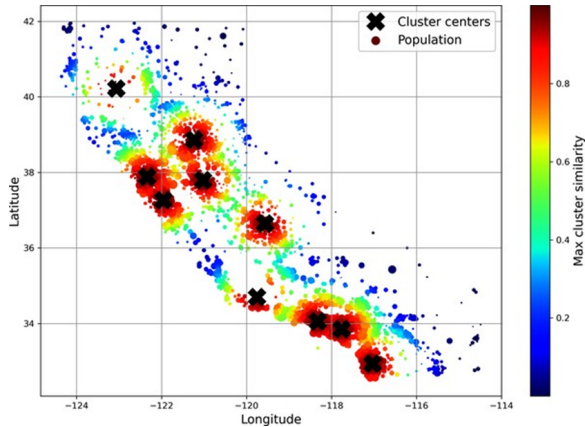
Biến đổi Đặc trưng bằng RBF

- Đối với phân phối có nhiều đỉnh (multimodal).
- Dùng **Radial Basis Function (RBF)** - Hàm cơ sở xuyên tâm. Đo khoảng cách (sự tương đồng) giữa một giá trị và một "đỉnh" (landmark) cố định.



Hình 2-18. Đặc trưng RBF (Đo khoảng cách tới 35)

Đặc trưng từ phân cụm bằng K-Means



Hình 2-19. Đặc trưng RBF với tâm cụm từ K-Means

Bộ biến đổi tùy chỉnh (Custom Transformers)

- Mặc dù Scikit-Learn cung cấp nhiều bộ biến đổi, ta vẫn cần tự viết mã để thực hiện logic riêng (ví dụ: tính số phòng trên mỗi hộ).
- Tạo các lớp (Class) tùy chỉnh tuân theo API của Scikit-Learn (kế thừa từ `BaseEstimator` và `TransformerMixin`).

Các pipeline biến đổi (Transformation Pipelines)

- Quá trình biến đổi dữ liệu yêu cầu thực hiện nhiều bước theo đúng thứ tự.
- Pipeline trong Scikit-Learn giúp liên kết các bước xử lý dữ liệu lại với nhau thành một luồng duy nhất.
- `ColumnTransformer`: Áp dụng các luồng xử lý khác nhau cho các loại cột khác nhau (VD: Cột số chạy qua `StandardScaler`, cột phân loại chạy qua `OneHotEncoder`).

3. Chọn, Huấn luyện, Tinh chỉnh & Triển khai

Huấn luyện trên Tập huấn luyện

- Bắt đầu với một mô hình đơn giản như **Hồi quy tuyến tính (Linear Regression)**.
- Nếu RMSE quá cao (ví dụ sai số 68,000 USD), nghĩa là mô hình bị **Underfitting** (chưa khớp).
- Thử nghiệm với một mô hình phức tạp hơn: **Cây quyết định (Decision Tree Regressor)**. Mô hình mạnh mẽ, có thể tìm ra các mối quan hệ phi tuyến tính phức tạp.

Đánh giá tốt hơn bằng kiểm định chéo (Cross-Validation)

- Sử dụng **K-fold Cross-Validation**.
- Chia ngẫu nhiên tập huấn luyện thành K tập con (fold) khác biệt.
- Huấn luyện và đánh giá mô hình K lần, mỗi lần chọn một fold khác nhau để đánh giá và K-1 fold còn lại để huấn luyện.
- Giúp biết chính xác mô hình tổng quát hóa tốt như thế nào mà không cần chạm vào tập kiểm thử thực (Test set).

Thử nghiệm với Rừng ngẫu nhiên (Random Forest)

- Xây dựng nhiều Cây quyết định trên các tập hợp con ngẫu nhiên của các đặc trưng, sau đó lấy trung bình dự đoán của chúng.
- Đây là một mô hình **Học tập hợp (Ensemble Learning)**.
- Thường cho kết quả vượt trội và là một trong những mô hình máy học mạnh nhất.

Tinh chỉnh mô hình

Tìm kiếm theo lưới (Grid Search)

- Sử dụng GridSearchCV của Scikit-Learn để tự động thử nghiệm tất cả các tổ hợp có thể có của các siêu tham số.
- Rất hữu ích khi khám phá một vài kết hợp, nhưng cực kỳ tốn thời gian tính toán nếu không gian tìm kiếm lớn.

Tìm kiếm ngẫu nhiên (Random Search)

- Dùng RandomizedSearchCV: thay vì thử nghiệm tất cả các kết hợp (như Grid Search), nó sẽ đánh giá một số lượng ngẫu nhiên nhất định.
- Cho phép điều chỉnh siêu tham số với hiệu suất tính toán tốt hơn nếu không gian tìm kiếm quá lớn (nhiều siêu tham số).

Các phương pháp tập hợp (Ensemble Methods)

- Một cách khác để tinh chỉnh hệ thống là cố gắng kết hợp các mô hình hoạt động tốt nhất.
- Một nhóm (ensemble) thường sẽ hoạt động tốt hơn so với mô hình cá nhân tốt nhất.
(Giống như quyết định của một hội đồng thường tốt hơn một cá nhân).

Phân tích các mô hình tốt nhất và lỗi của chúng

- Có thể đạt được những cái nhìn sâu sắc bằng cách kiểm tra các mô hình tốt nhất.
- Mô hình RandomForest có thể cho biết mức độ quan trọng tương đối của từng đặc trưng.
- Dựa vào thông tin này, bạn có thể cân nhắc loại bỏ một số đặc trưng ít hữu ích nhất.

Đánh giá hệ thống trên tập kiểm thử

- Khi đã hài lòng với mô hình cuối cùng, đây là lúc kiểm tra hiệu suất trên **Test Set**.
- Chạy tập kiểm thử qua `full_pipeline` để chuyển đổi dữ liệu. **Lưu ý: Chỉ gọi `transform()`, không gọi `fit()`.**
- Hiệu suất thường sẽ hơi tệ hơn so với trên tập xác thực (do mô hình đã được tinh chỉnh cho tập xác thực). Đừng cố tình điều chỉnh siêu tham số nữa!

Triển khai, Giám sát và Bảo trì hệ thống

Triển khai mô hình (Model Deployment)

- Lưu trữ mô hình đã huấn luyện bằng cách sử dụng các thư viện như joblib.
- Tải mô hình lên môi trường sản xuất (Production).
- Có thể đóng gói bằng Container (Docker) và triển khai lên Cloud.

Kiến trúc Triển khai (Ví dụ)



Hình 2-20. Triển khai mô hình học máy thành dịch vụ web (Web API)

Giám sát hệ thống (Monitoring)

- Theo thời gian, dữ liệu thực tế thay đổi dẫn đến hiện tượng suy giảm hiệu suất (Model Rot / Data Drift).
- Cần theo dõi sự chênh lệch giữa dự đoán của hệ thống và dữ liệu thực tế mới nhất.
- Thiết lập cảnh báo để đội ngũ xử lý khi độ chính xác giảm xuống dưới một ngưỡng nhất định.

Tự động hóa quá trình bảo trì và cập nhật

- Thu thập dữ liệu mới thường xuyên và tự động gán nhãn cho nó (nếu có thể).
- Tự động đánh giá các mô hình mới. Nếu hiệu suất tốt hơn mô hình cũ, tự động đẩy (deploy) phiên bản mới lên máy chủ.
- Giữ lại các bản sao lưu để có thể quay lại (rollback) phiên bản trước nhanh chóng nếu mô hình mới gây ra lỗi nghiêm trọng.

Tổng kết Chương 2

- Quá trình xây dựng ứng dụng học máy là một chu trình **khép kín (End-to-End)**: Từ dữ liệu thô → Phân tích & Chuẩn bị → Huấn luyện → Triển khai → Bảo trì.
- Hầu hết công sức của chuyên gia dữ liệu là ở khâu **Chuẩn bị dữ liệu** và hiểu bài toán, thay vì chọn các thuật toán tinh vi.
- Một pipeline dữ liệu vững chắc là chìa khóa thành công.